

Data Science / ML Curriculum

Part of the [training document series](#). This folder is a self-contained curriculum covering data engineering, feature engineering, classical regression modeling, and time-series forecasting for panel/time-series data — the concepts and libraries needed to take a raw dataset all the way to a compared, validated set of models and a multi-period forecast.

Reading order

1. [EDA & Data Engineering](#) — profiling a raw dataset, filtering it responsibly, exploratory data analysis, and the data-quality discipline needed before any modeling begins.
2. [Feature Engineering](#) — turning cleaned data into model-ready inputs: time-based, lag, rolling, growth, and ratio features; scaling; categorical encoding.
3. [Regression Models](#) — Naive baseline, Linear Regression, and Random Forest: the concepts, the scikit-learn API, key parameters, evaluation metrics, and the bias-variance trade-off that ties them together.
4. [Time-Series Forecasting](#) — Exponential Smoothing (ETS/Holt's Damped Trend), multi-step recursive forecasting, and what-if scenario modeling, plus where this fits relative to ARIMA and other forecasting approaches.
5. [Multi-Source Pipelines](#) — what changes when several external sources are combined and refreshed on a schedule: one ingestion module per source, validation before publishing, provenance and licence as data, a harmonised layer, honest statistical relationships (HAC errors, block bootstrap, holdouts), scenario translation, and unattended operation.

Documents 1–4 are the core curriculum, built on one dataset; document 5 is the extension for a product that has to keep several sources current. Each document ends with its own **Core Libraries**, **Testing & Validation**, and (where relevant) **Deployment** sections, so it can be read and referenced independently once you've been through the series once.

Prerequisites

Comfort with Python and basic pandas (reading a CSV, indexing a DataFrame) is assumed. No prior machine learning or statistics background is assumed — each concept is introduced before the code that implements it.

See also

- [Architecture Overview](#) for how this pipeline's output typically gets served and displayed
- [Python API Backend](#) for how to expose these results over HTTP